
Intervals and Galois Connections

Lecture 5 — Static Analysis by Abstract Interpretation

David Pichardie

Fall 2026

Soundness is not enough

A monotone concretization function $\gamma : L^\# \rightarrow L$ was enough to state and prove soundness of each abstract operator. But an analysis that always returns $\top$ is sound as well.

Two precision questions remain:

- given a concrete property, does a **most precise** sound abstract description exist?
- given a concrete operator, can we **derive** its best sound abstract counterpart rather than guess it?

Next

Intervals provide the running numerical abstraction; Galois connections provide the calculation principle.

Numerical abstraction with intervals

Galois connections and insertions

Best correct approximations

Calculational design of interval transformers

§ 1

Numerical abstraction with intervals

Why intervals?

Signs retain only qualitative information.

concrete property	signs	intervals
$0 \leq x \leq 10$	$\top$	$[0, 10]$
$x \geq 7$	$+$	$[7, +\infty]$
$x = -3$	$-$	$[-3, -3]$

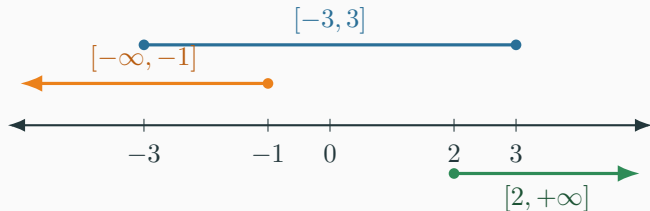
The trade-off

Intervals remember numerical bounds while remaining cheap: an abstract value is represented by two endpoints.

The interval domain

Lower bounds range over $\mathbb{Z} \cup \{-\infty\}$ and upper bounds over $\mathbb{Z} \cup \{+\infty\}$.

$$\mathbb{I} \triangleq \{\perp_I\} \cup \{[\ell, u] \mid \ell \in \mathbb{Z} \cup \{-\infty\}, u \in \mathbb{Z} \cup \{+\infty\}, \ell \leq u\}.$$



$\perp_I$ means impossible, $\top_I = [-\infty, +\infty]$ means unknown.

The meaning of an interval is the set of integers between its endpoints:

$$\gamma_I : \mathbb{I} \longrightarrow \mathcal{P}(\mathbb{Z}),$$

$$\gamma_I(\perp_I) \triangleq \emptyset, \quad \gamma_I([\ell, u]) \triangleq \{n \in \mathbb{Z} \mid \ell \leq n \leq u\}.$$

Example 1

$$\gamma_I([-2, 3]) = \{-2, -1, 0, 1, 2, 3\}, \quad \gamma_I([4, +\infty]) = \{n \in \mathbb{Z} \mid n \geq 4\}.$$

A set $S \subseteq \mathbb{Z}$ is represented exactly iff it is *integer-convex*:

$$\forall m, n \in S, \forall k \in \mathbb{Z}, \quad m \leq k \leq n \implies k \in S.$$

Precision order and lattice operations

The abstract order follows inclusion of concretizations:

$$\perp_I \sqsubseteq_I I, \quad [\ell_1, u_1] \sqsubseteq_I [\ell_2, u_2] \iff \ell_2 \leq \ell_1 \wedge u_1 \leq u_2.$$

For non-bottom intervals:

$$\begin{aligned} [\ell_1, u_1] \sqcup_I [\ell_2, u_2] &= [\min(\ell_1, \ell_2), \max(u_1, u_2)], \\ [\ell_1, u_1] \sqcap_I [\ell_2, u_2] &= \begin{cases} [\max(\ell_1, \ell_2), \min(u_1, u_2)] & \max(\ell_1, \ell_2) \leq \min(u_1, u_2), \\ \perp_I & \text{otherwise.} \end{cases} \end{aligned}$$

Geometric reading

Join is the smallest interval containing both operands; meet is their intersection.

Exercise — interval mechanics

Let

$$I = [-3, 2], \quad J = [1, 5], \quad K = [-\infty, 0].$$

Compute:

$$I \sqcup_I J, \quad I \sqcap_I J, \quad J \sqcap_I K,$$

and decide whether $I \sqsubseteq_I K$, $K \sqsubseteq_I I$, or neither.

Exercise — interval mechanics

Let

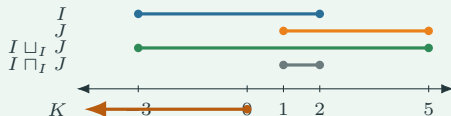
$$I = [-3, 2], \quad J = [1, 5], \quad K = [-\infty, 0].$$

Compute:

$$I \sqcup_I J, \quad I \cap_I J, \quad J \cap_I K,$$

and decide whether $I \sqsubseteq_I K$, $K \sqsubseteq_I I$, or neither.

Solution



$$I \sqcup_I J = [-3, 5],$$

$$I \cap_I J = [1, 2],$$

$$J \cap_I K = \perp_I.$$

I and K are incomparable: each contains values absent from the other.

Introducing an abstraction function

The abstraction function returns the smallest interval containing a set:

$$\alpha_I : \mathcal{P}(\mathbb{Z}) \longrightarrow \mathbb{I},$$
$$\alpha_I(S) \triangleq \begin{cases} \perp_I & S = \emptyset, \\ [\inf S, \sup S] & S \neq \emptyset, \end{cases}$$

where the infimum and supremum are taken in $\mathbb{Z} \cup \{-\infty, +\infty\}$.

Example 2

$$\alpha_I(\{-4, 1, 7\}) = [-4, 7], \quad \alpha_I(\{2n \mid n \in \mathbb{Z}\}) = [-\infty, +\infty].$$

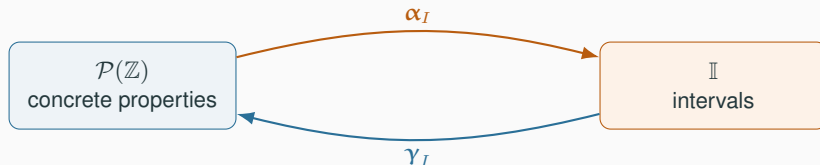
The second abstraction fills every odd-numbered hole.

Fundamental properties of α_I and γ_I

Both functions are monotone, and for every $S \subseteq \mathbb{Z}$ and $I \in \mathbb{I}$:

$$S \subseteq \gamma_I(\alpha_I(S)),$$
$$\alpha_I(\gamma_I(I)) = I,$$

$\alpha_I(S)$ is a sound abstraction of S ,
the best abstraction of $\gamma_I(I)$ is I .



Proofs

If $S = \emptyset$, the first inclusion is immediate. Otherwise, for every $x \in S$, $\inf S \leq x \leq \sup S$; hence $x \in \gamma_I(\alpha_I(S))$. For the second equality,

$$\alpha_I(\gamma_I(\perp_I)) = \alpha_I(\emptyset) = \perp_I, \quad \alpha_I(\gamma_I([\ell, u])) = [\inf \gamma_I([\ell, u]), \sup \gamma_I([\ell, u])] = [\ell, u].$$

Exercise — abstraction may add values

Give a finite set $S \subseteq \mathbb{Z}$ such that

$$S \subsetneq \gamma_I(\alpha_I(S)).$$

Compute $\alpha_I(S)$ and identify the concrete values introduced by the abstraction.

Exercise — abstraction may add values

Give a finite set $S \subseteq \mathbb{Z}$ such that

$$S \subsetneq \gamma_I(\alpha_I(S)).$$

Compute $\alpha_I(S)$ and identify the concrete values introduced by the abstraction.

Solution

Take $S = \{-1, 1\}$. Then

$$\alpha_I(S) = [-1, 1], \quad \gamma_I(\alpha_I(S)) = \{-1, 0, 1\}.$$

Hence

$$\{-1, 1\} \subsetneq \{-1, 0, 1\};$$

the abstraction introduces the spurious value 0.

More generally, the inclusion is strict exactly when S is not integer-convex.

§2

Galois connections and insertions

Definition 3 (Galois connection)

Let $(C, \sqsubseteq_C)$ and $(A, \sqsubseteq_A)$ be partially ordered sets. Monotone maps

$$C \begin{matrix} \xrightarrow{\alpha} \\ \xleftarrow{\gamma} \end{matrix} A$$

form a *Galois connection* when, for every $c \in C$ and $a \in A$,

$$\boxed{\alpha(c) \sqsubseteq_A a \iff c \sqsubseteq_C \gamma(a).}$$

Reading the equivalence

“ a soundly describes c ” is equivalent to “the canonical abstraction $\alpha(c)$ is more precise than a ”.

Here α is the *abstraction function* and γ the *concretization function*.

Exercise — intervals form a Galois connection

Prove, for every $S \subseteq \mathbb{Z}$ and $I \in \mathbb{I}$,

$$\alpha_I(S) \sqsubseteq_I I \iff S \subseteq \gamma_I(I).$$

Treat $S = \emptyset$ first, then write $I = [\ell, u]$.

Exercise — intervals form a Galois connection

Prove, for every $S \subseteq \mathbb{Z}$ and $I \in \mathbb{I}$,

$$\alpha_I(S) \sqsubseteq_I I \iff S \subseteq \gamma_I(I).$$

Treat $S = \emptyset$ first, then write $I = [\ell, u]$.

Solution

If $S = \emptyset$, both sides hold. If $S \neq \emptyset$ and $I = \perp_I$, both sides fail. Otherwise, write $I = [\ell, u]$:

$$\begin{aligned} \alpha_I(S) \sqsubseteq_I [\ell, u] &\iff [\inf S, \sup S] \sqsubseteq_I [\ell, u] \\ &\iff \ell \leq \inf S \wedge \sup S \leq u \\ &\iff \forall n \in S, \ell \leq n \leq u \\ &\iff S \subseteq \gamma_I([\ell, u]). \end{aligned}$$

Two properties of a Galois connection

For monotone $\alpha : C \rightarrow A$ and $\gamma : A \rightarrow C$, the defining equivalence is equivalent to the two inequalities

$$\boxed{\text{id}_C \sqsubseteq_C \gamma \circ \alpha} \quad \text{and} \quad \boxed{\alpha \circ \gamma \sqsubseteq_A \text{id}_A}.$$

Sound abstraction

$$c \sqsubseteq_C \gamma(\alpha(c)).$$

Thus $\alpha(c)$ is a sound abstraction of c .

Re-abstraction cannot lose precision

$$\alpha(\gamma(a)) \sqsubseteq_A a.$$

Re-abtracting the meaning of a can only make its description more precise.

Why these two inequalities suffice

Assume monotonicity and the two preceding inequalities.

$$\begin{aligned}\alpha(c) \sqsubseteq_A a &\implies \gamma(\alpha(c)) \sqsubseteq_C \gamma(a) \\ &\implies c \sqsubseteq_C \gamma(a) \\ c \sqsubseteq_C \gamma(a) &\implies \alpha(c) \sqsubseteq_A \alpha(\gamma(a)) \\ &\implies \alpha(c) \sqsubseteq_A a\end{aligned}$$

monotonicity of γ
sound abstraction,
monotonicity of α
abstract round trip.

Conversely, assume the defining equivalence. Reflexivity gives

$$\begin{aligned}\underbrace{\alpha(c) \sqsubseteq_A \alpha(c)}_{\text{reflexivity}} &\implies c \sqsubseteq_C \gamma(\alpha(c)) \\ \alpha(\gamma(a)) \sqsubseteq_A a &\longleftarrow \underbrace{\gamma(a) \sqsubseteq_C \gamma(a)}_{\text{reflexivity}}\end{aligned}$$

take $a = \alpha(c)$,

take $c = \gamma(a)$.

Definition 4 (Galois insertion)

A Galois connection is a *Galois insertion* when

$$\alpha \circ \gamma = \text{id}_A.$$

Every abstract element is then the canonical abstraction of its meaning:

$$\alpha(\gamma(a)) = a.$$

Intervals

By the equality proved earlier, $\alpha_I(\gamma_I(I)) = I$; hence the interval abstraction is a Galois insertion:

$$\mathcal{P}(\mathbb{Z}) \begin{matrix} \xrightarrow{\alpha_I} \\ \xleftarrow{\gamma_I} \end{matrix} \mathbb{I}.$$

Equivalently, in a Galois connection, γ is injective (or α is surjective).

Why Galois insertions matter

In a Galois insertion, the monotonicity implication becomes an equivalence:

$$a \sqsubseteq_A b \iff \gamma(a) \sqsubseteq_C \gamma(b).$$

The forward implication is monotonicity of γ . Conversely,

$$\gamma(a) \sqsubseteq_C \gamma(b) \implies \alpha(\gamma(a)) \sqsubseteq_A \alpha(\gamma(b)) \implies a \sqsubseteq_A b,$$

because $\alpha \circ \gamma = \text{id}_A$.

Intervals

The abstract order tests concrete inclusion exactly:

$$I \sqsubseteq_I J \iff \gamma_I(I) \subseteq \gamma_I(J).$$

Sign states in Lecture 4

Let

$$A = [x \mapsto \perp, y \mapsto +], \quad B = [x \mapsto \perp, y \mapsto -].$$

Then $\gamma_\Sigma(A) = \gamma_\Sigma(B) = \emptyset$, but $A \not\sqsubseteq^\# B$. Thus concrete inclusion does not characterize the pointwise order.

To avoid several abstract representations of the empty set, use normalized environments:

$$\Sigma_I^\# \triangleq \{\perp_\Sigma\} \cup (\text{Var} \rightarrow \mathbb{I}_+), \quad \mathbb{I}_+ \triangleq \mathbb{I} \setminus \{\perp_I\}.$$

Order environments pointwise, with $\perp_\Sigma \sqsubseteq_\Sigma A$ for every A .

$$\begin{aligned} \gamma_\Sigma(\perp_\Sigma) &\triangleq \emptyset, \\ \gamma_\Sigma(A) &\triangleq \{\sigma \in \Sigma \mid \forall x, \sigma(x) \in \gamma_I(A(x))\}, \\ \alpha_\Sigma(S)(x) &\triangleq \alpha_I(\{\sigma(x) \mid \sigma \in S\}) \quad (S \neq \emptyset), \\ \alpha_\Sigma(\emptyset) &\triangleq \perp_\Sigma. \end{aligned}$$

We now prove that, for finite Var , these maps form a Galois insertion.

Theorem 5

$(\alpha_\Sigma, \gamma_\Sigma)$ is a Galois connection.

Proof. Both maps are monotone. For the defining equivalence, distinguish three cases:

1. If $S = \emptyset$, both sides hold.
2. If $S \neq \emptyset$ and $A = \perp_\Sigma$, both sides are false: $\alpha_\Sigma(S) \neq \perp_\Sigma$ and $S \not\subseteq \gamma_\Sigma(A) = \emptyset$.
3. If $S \neq \emptyset$ and $A \neq \perp_\Sigma$, then

$$\begin{aligned}\alpha_\Sigma(S) \sqsubseteq_\Sigma A &\iff \forall x, \alpha_I(\{\sigma(x) \mid \sigma \in S\}) \sqsubseteq_I A(x) \\ &\iff \forall x, \{\sigma(x) \mid \sigma \in S\} \subseteq \gamma_I(A(x)) \\ &\iff \forall \sigma \in S, \forall x, \sigma(x) \in \gamma_I(A(x)) \\ &\iff S \subseteq \gamma_\Sigma(A).\end{aligned}$$

Hence $(\alpha_\Sigma, \gamma_\Sigma)$ is a Galois connection.

Interval states form a Galois insertion

Theorem 6

For finite Var , the preceding Galois connection is a Galois insertion.

Proof.

1. First characterize the empty abstract state:

$$\gamma_{\Sigma}(A) = \emptyset \iff A = \perp_{\Sigma}.$$

The reverse implication is by definition. If $A \neq \perp_{\Sigma}$, then $A(x) \neq \perp_I$ for every x . Since the interval connection is an insertion, $\gamma_I(A(x)) \neq \emptyset$; choose $n_x \in \gamma_I(A(x))$ for each variable. The state $\lambda x. n_x$ belongs to $\gamma_{\Sigma}(A)$.

2. By definition,

$$\alpha_{\Sigma}(\gamma_{\Sigma}(\perp_{\Sigma})) = \alpha_{\Sigma}(\emptyset) = \perp_{\Sigma}.$$

3. Now let $A \neq \perp_{\Sigma}$ and fix x :

$$\begin{aligned} \alpha_{\Sigma}(\gamma_{\Sigma}(A))(x) &= \alpha_I(\{\sigma(x) \mid \sigma \in \gamma_{\Sigma}(A)\}) && \text{by definition of } \alpha_{\Sigma}, \text{ since } \gamma_{\Sigma}(A) \neq \emptyset \\ &= \alpha_I(\{\sigma(x) \mid \forall y, \sigma(y) \in \gamma_I(A(y))\}) && \text{by definition of } \gamma_{\Sigma}, \text{ since } A \neq \perp_{\Sigma} \\ &= \alpha_I(\gamma_I(A(x))) && \subseteq: \text{specialize } \forall y \text{ to } y = x. \\ &&& \supseteq: \text{given } n \in \gamma_I(A(x)), \text{ choose} \\ &&& n_y \in \gamma_I(A(y)) \text{ for } y \neq x; \text{ the state} \\ &&& \sigma_n(x) = n, \sigma_n(y) = n_y \text{ belongs to } \gamma_{\Sigma}(A). \\ &= A(x) && \text{because the interval connection is an insertion.} \end{aligned}$$

Hence $\alpha_{\Sigma} \circ \gamma_{\Sigma} = \text{id}_{\Sigma_I^{\#}}$.

Injectivity and dead branches

In a Galois insertion, γ is injective: each concrete property has a unique abstract representation.

Lecture 4: sign states

Let

$$D = [x \mapsto \perp, y \mapsto +], \quad \gamma_{\Sigma}(D) = \emptyset.$$

A pointwise update revives this dead branch:

$$\text{assign}^{\sharp}(x, 0, D) = [x \mapsto 0, y \mapsto +] = D'.$$

At the merge with $L = [x \mapsto 0, y \mapsto -]$,

$$D' \sqcup^{\sharp} L = [x \mapsto 0, y \mapsto \top].$$

The dead branch makes y imprecise.

Normalized interval states

We use $\perp_{\Sigma}$ as the unique representation of $\emptyset$, and assignment is strict:

$$\text{assign}_I^{\sharp}(x, 0, \perp_{\Sigma}) = \perp_{\Sigma}.$$

For every live interval state L_I ,

$$\perp_{\Sigma} \sqcup_{\Sigma} L_I = L_I.$$

The dead branch cannot affect the join.

Why injectivity helps

A unique representation of $\emptyset$, together with strict transformers, prevents dead branches from being revived.

§3

Best correct approximations

The best sound description of a property

Let $C \overset{\alpha}{\underset{\gamma}{\rightleftharpoons}} A$ be a Galois connection.

1. An abstract element a soundly describes c when

$$c \sqsubseteq_C \gamma(a).$$

2. $\alpha(c)$ is such a sound abstraction because

$$c \sqsubseteq_C \gamma(\alpha(c)).$$

3. For any other sound abstraction a , the defining equivalence gives

$$c \sqsubseteq_C \gamma(a) \iff \alpha(c) \sqsubseteq_A a.$$

Theorem 7

Among all sound abstract descriptions of c , $\boxed{\alpha(c)}$ is the most precise one.

For intervals, the best sound description of a set is therefore its interval hull.

Sound abstract transformers

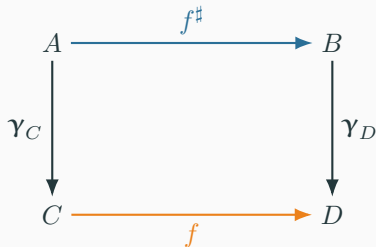
Consider two Galois connections

$$C \begin{smallmatrix} \xrightarrow{\alpha_C} \\ \xleftarrow{\gamma_C} \end{smallmatrix} A \quad \text{and} \quad D \begin{smallmatrix} \xrightarrow{\alpha_D} \\ \xleftarrow{\gamma_D} \end{smallmatrix} B,$$

and a monotone concrete transformer $f : C \rightarrow D$.

An abstract transformer $f^\# : A \rightarrow B$ is sound when

$$f \circ \gamma_C \sqsubseteq_D \gamma_D \circ f^\#.$$



Calculating the best correct approximation

Theorem 8 (Best correct approximation)

The most precise sound abstract transformer exists and is

$$f^{\text{best}} \triangleq \alpha_D \circ f \circ \gamma_C.$$

Soundness

The sound-abstraction property on D gives

$$f \circ \gamma_C \sqsubseteq_D \gamma_D \circ \alpha_D \circ f \circ \gamma_C = \gamma_D \circ f^{\text{best}}.$$

Optimality

If $f^\#$ is sound, monotonicity and the round-trip inequality give

$$f^{\text{best}} = \alpha_D \circ f \circ \gamma_C \sqsubseteq_B \alpha_D \circ \gamma_D \circ f^\# \sqsubseteq_B f^\#.$$

The calculation principle

The preceding theorem already gives the optimal mathematical specification:

$$f^{\text{best}}(a) = \alpha_D(f(\gamma_C(a))).$$

Derive an executable formula by equational reasoning:

$$\begin{aligned} f^{\text{best}}(a) &= \alpha_D(f(\gamma_C(a))) && \text{best mathematical specification} \\ &= \dots && \text{unfold definitions and simplify} \\ &= F(a) && \text{executable formula.} \end{aligned}$$

Exact calculation

If the derivation uses only equalities, then $F = f^{\text{best}}$: the resulting formula is sound and optimal.

If exact simplification is impossible, replace a step by a sound upper approximation; soundness remains, but optimality may be lost. This is *calculational design*.

Several arguments

For

$$f : C_1 \times \cdots \times C_n \longrightarrow D,$$

with one Galois connection per argument and one for the result, the same calculation gives

$$f^{\text{best}}(a_1, \dots, a_n) = \alpha_D(f(\gamma_1(a_1), \dots, \gamma_n(a_n))).$$

For numerical operations

The concrete operation is lifted to sets, for example

$$X + Y \triangleq \{x + y \mid x \in X, y \in Y\}.$$

Its best interval implementation is $I +^{\text{best}} J = \alpha_I(\gamma_I(I) + \gamma_I(J))$.

§ 4

Computational design of interval transformers

Deriving interval addition

Let $I = [\ell_1, u_1]$ and $J = [\ell_2, u_2]$. Starting from the best correct approximation:

$$\begin{aligned} I +^{\text{best}} J &= \alpha_I(\gamma_I(I) + \gamma_I(J)) && \text{definition of the BCA} \\ &= \left[\inf_{\substack{x \in I \\ y \in J}} (x + y), \sup_{\substack{x \in I \\ y \in J}} (x + y) \right] && \text{definition of } \alpha_I \\ &= [\ell_1 + \ell_2, u_1 + u_2] && \text{monotonicity of integer addition.} \end{aligned}$$

Thus the executable definition is

$$\boxed{[\ell_1, u_1] +^{\#} [\ell_2, u_2] \triangleq [\ell_1 + \ell_2, u_1 + u_2].}$$

If either operand is $\perp_I$, the result is $\perp_I$.

Negation and subtraction

Negation reverses the endpoints:

$$-\sharp[\ell, u] \triangleq [-u, -\ell].$$

Subtraction is addition of the negated second operand:

$$\begin{aligned} [\ell_1, u_1] - \sharp [\ell_2, u_2] &\triangleq [\ell_1, u_1] + \sharp [-u_2, -\ell_2] \\ &= [\ell_1 - u_2, u_1 - \ell_2]. \end{aligned}$$

Example 9

$$[-2, 4] - [1, 3] = [-5, 3].$$

Interval multiplication

The product is not monotone in each numerical argument over all of $\mathbb{Z}$, so all four corners matter:

$$P \triangleq \{\ell_1 \ell_2, \ell_1 u_2, u_1 \ell_2, u_1 u_2\}.$$

Calculating the interval hull gives

$$[\ell_1, u_1] \times^\# [\ell_2, u_2] \triangleq [\min P, \max P].$$

Example 10

$$[-2, 3] \times^\# [4, 7] = [\min\{-8, -14, 12, 21\}, \max\{-8, -14, 12, 21\}] = [-14, 21].$$

With infinite endpoints, use the limiting conventions for interval bounds, in particular $0 \cdot (\pm\infty) = 0$. Bottom remains absorbing.

Exercise — calculate interval operations

Compute the best interval results:

1. $[-3, 5] +^{\#} [2, 4];$
2. $[-3, 5] -^{\#} [2, 4];$
3. $[-3, 5] \times^{\#} [2, 4];$
4. $[-\infty, -1] \times^{\#} [0, 3].$

Exercise — calculate interval operations

Compute the best interval results:

1. $[-3, 5] +^{\#} [2, 4]$;
2. $[-3, 5] -^{\#} [2, 4]$;
3. $[-3, 5] \times^{\#} [2, 4]$;
4. $[-\infty, -1] \times^{\#} [0, 3]$.

Solution

$$\begin{aligned}[-3, 5] +^{\#} [2, 4] &= [-1, 9], \\[-3, 5] -^{\#} [2, 4] &= [-7, 3], \\[-3, 5] \times^{\#} [2, 4] &= [-12, 20], \\[-\infty, -1] \times^{\#} [0, 3] &= [-\infty, 0].\end{aligned}$$

Abstract evaluation of expressions

As for signs, define abstract evaluation recursively. Set $\text{eval}_I^\sharp(a, \perp_\Sigma) \triangleq \perp_I$. For $A \neq \perp_\Sigma$, define recursively:

$$\text{eval}_I^\sharp(n, A) \triangleq [n, n],$$

$$\text{eval}_I^\sharp(x, A) \triangleq A(x),$$

$$\text{eval}_I^\sharp(a_1 + a_2, A) \triangleq \text{eval}_I^\sharp(a_1, A) +^\sharp \text{eval}_I^\sharp(a_2, A),$$

$$\text{eval}_I^\sharp(a_1 - a_2, A) \triangleq \text{eval}_I^\sharp(a_1, A) -^\sharp \text{eval}_I^\sharp(a_2, A),$$

$$\text{eval}_I^\sharp(a_1 \times a_2, A) \triangleq \text{eval}_I^\sharp(a_1, A) \times^\sharp \text{eval}_I^\sharp(a_2, A).$$

Theorem 11 (Soundness)

$$\forall \sigma \in \gamma_\Sigma(A), \quad \llbracket a \rrbracket \sigma \in \gamma_I(\text{eval}_I^\sharp(a, A)).$$

The proof is by structural induction on a and uses only the concretization functions γ_Σ and γ_I .

As for signs, define assignment by updating the assigned variable:

$$\text{assign}_I^\sharp(x, a, A) \triangleq \begin{cases} \perp_\Sigma & A = \perp_\Sigma, \\ A[x \mapsto \text{eval}_I^\sharp(a, A)] & A \neq \perp_\Sigma. \end{cases}$$

Theorem 12 (Soundness)

$$\llbracket x := a \rrbracket(\gamma_\Sigma(A)) \subseteq \gamma_\Sigma(\text{assign}_I^\sharp(x, a, A)).$$

Proof. The case $A = \perp_\Sigma$ is immediate. Otherwise, take $\sigma' \in \llbracket x := a \rrbracket(\gamma_\Sigma(A))$. Then $\sigma' = \sigma[x \mapsto \llbracket a \rrbracket \sigma]$ for some $\sigma \in \gamma_\Sigma(A)$. Expression soundness gives

$$\sigma'(x) = \llbracket a \rrbracket \sigma \in \gamma_I(\text{eval}_I^\sharp(a, A)).$$

For every $y \neq x$, $\sigma'(y) = \sigma(y) \in \gamma_I(A(y))$. Hence $\sigma' \in \gamma_\Sigma(\text{assign}_I^\sharp(x, a, A))$.

Exercise — abstract assignment

Let

$$A = [x \mapsto [-2, 4], y \mapsto [1, 3], z \mapsto \top_I].$$

Compute the abstract state obtained after executing

$$z := 2 \times x - y.$$

Solution

$$\begin{aligned}\text{eval}_I^\sharp(2 \times x, A) &= [2, 2] \times^\sharp [-2, 4] = [-4, 8], \\ \text{eval}_I^\sharp(2 \times x - y, A) &= [-4, 8] -^\sharp [1, 3] = [-7, 7], \\ \text{assign}_I^\sharp(z, 2 \times x - y, A) &= A[z \mapsto [-7, 7]].\end{aligned}$$

Exercise — assignment and dependencies

Initially,

$$A = [x \mapsto [0, 2], y \mapsto [0, 2], z \mapsto \top_I, w \mapsto \top_I].$$

Compute the interval states after

$$z := x - y; \quad w := z + y.$$

Then simplify the concrete expression assigned to w . What is its best interval, and where was precision lost?

Exercise — assignment and dependencies

Initially,

$$A = [x \mapsto [0, 2], y \mapsto [0, 2], z \mapsto \top_I, w \mapsto \top_I].$$

Compute the interval states after

$$z := x - y; \quad w := z + y.$$

Then simplify the concrete expression assigned to w . What is its best interval, and where was precision lost?

Solution

$$z \in [0, 2] -^\# [0, 2] = [-2, 2],$$

$$w \in [-2, 2] +^\# [0, 2] = [-2, 4].$$

But $w = (x - y) + y = x$, so its best interval is $[0, 2]$. The environment forgets the relation $z = x - y$ before the second assignment.

A baseline from Lecture 4

We could reuse the abstract-filter construction developed for signs. It would remain sound, but would miss many interval-specific refinements.

Project 2: do better

Propagate a desired interval backward through expressions, then use it to refine both sides of a comparison.

Important operators

$$\begin{aligned} \text{back}_I^\sharp &: \text{AExp} \times \mathbb{I} \times \Sigma_I^\sharp \rightarrow \Sigma_I^\sharp && (\text{backward_aexp}), \\ \text{filter_cmp}_I^\sharp &: \text{CmpOp} \times \text{AExp} \times \text{AExp} \times \Sigma_I^\sharp \rightarrow \Sigma_I^\sharp && (\text{filter_cmp}), \\ \text{filter}_I^\sharp &: \text{BExp} \times \Sigma_I^\sharp \rightarrow \Sigma_I^\sharp && (\text{student_filter}). \end{aligned}$$

The required result remains sound and reductive:

$$\{\sigma \in \gamma_\Sigma(A) \mid \llbracket b \rrbracket \sigma = \text{tt}\} \subseteq \gamma_\Sigma(\text{filter}_I^\sharp(b, A)), \quad \text{filter}_I^\sharp(b, A) \sqsubseteq_\Sigma A.$$

The recursive interval filter

After pushing negations down to comparisons, define

$$\text{filter}_I^\sharp(b, A) \triangleq \begin{cases} A & b = \text{true}, \\ \perp_\Sigma & b = \text{false}, \\ \text{filter_cmp}_I^\sharp(p, A) & b = p, \\ \text{filter}_I^\sharp(b_2, \text{filter}_I^\sharp(b_1, A)) & b = b_1 \wedge b_2, \\ \text{filter}_I^\sharp(b_1, A) \sqcup_\Sigma \text{filter}_I^\sharp(b_2, A) & b = b_1 \vee b_2. \end{cases}$$

Project 2

The implementation follows these three layers:

1. evaluate expressions forward with intervals (`eval`);
2. propagate desired intervals backward through expressions (`backward_aexp`);
3. filter atomic comparisons (`filter_cmp`), then combine them recursively over booleans (`student_filter`).

Deadline: send me `interval.ml` by midnight on Thursday, 24 September.

What should `filter_cmp` compute?

Informally, $\text{filter_cmp}_I^\sharp(\bowtie, a_1, a_2, A)$ must refine the intervals attached to each variable in A , assuming that the values v_1 and v_2 computed by a_1 and a_2 satisfy $v_1 \bowtie v_2$.

By the Galois-connection equivalence from slide 20, this means:

$$\alpha_\Sigma(\{\sigma \in \gamma_\Sigma(A) \mid \llbracket a_1 \rrbracket \sigma \bowtie \llbracket a_2 \rrbracket \sigma\}) \sqsubseteq_\Sigma \text{filter_cmp}_I^\sharp(\bowtie, a_1, a_2, A) \sqsubseteq_\Sigma A.$$

Example — filtering $x + y \leq z$

Start from

$$A = [x \mapsto [4, 10], y \mapsto [2, 4], z \mapsto [3, 8]].$$

1. Since $z \leq 8$, the guard requires $x + y \leq 8$.
2. Since $y \geq 2$, we obtain $x \leq 6$:

$$x \in [4, 10] \sqcap_I [-\infty, 8 - 2] = [4, 6].$$

3. With the refined bound $x \geq 4$ and $y \geq 2$, we have $x + y \geq 6$; hence

$$z \in [3, 8] \sqcap_I [6, +\infty] = [6, 8].$$

No further refinement of y is possible, so

$$\text{filter_cmp}_I^\#(\leq, x + y, z, A) = [x \mapsto [4, 6], y \mapsto [2, 4], z \mapsto [6, 8]].$$

Backward refinement of expressions

To implement $\text{filter_cmp}_I^\sharp$, we need a backward operator on arithmetic expressions.

Informally, $\text{back}_I^\sharp(a, J, A)$ must refine the intervals attached to each variable in A , assuming that the value v computed by a belongs to J .

Formally,

$$\alpha_\Sigma(\{\sigma \in \gamma_\Sigma(A) \mid \llbracket a \rrbracket \sigma \in \gamma_I(J)\}) \sqsubseteq_\Sigma \text{back}_I^\sharp(a, J, A) \sqsubseteq_\Sigma A.$$

Example 13

For $a = a_1 - a_2$:

$$\begin{aligned} I_2 &\triangleq \text{eval}_I^\sharp(a_2, A), \\ A_1 &\triangleq \text{back}_I^\sharp(a_1, J +^\sharp I_2, A), \\ \text{back}_I^\sharp(a_1 - a_2, J, A) &\triangleq \text{back}_I^\sharp(a_2, \text{eval}_I^\sharp(a_1, A_1) -^\sharp J, A_1). \end{aligned}$$

Example — refining both variables in $x - y$

Let

$$A = [x \mapsto [0, 10], y \mapsto [0, 10]], \quad J = [2, 4].$$

We assume that $x - y \in J$.

First refine x using the current interval for y :

$$\begin{aligned} I_2 &= \text{eval}_I^\sharp(y, A) = [0, 10], \\ J +^\sharp I_2 &= [2, 4] +^\sharp [0, 10] = [2, 14], \\ A_1 &= \text{back}_I^\sharp(x, [2, 14], A) = [x \mapsto [2, 10], y \mapsto [0, 10]]. \end{aligned}$$

Then use the refined interval for x to refine y :

$$\text{eval}_I^\sharp(x, A_1) -^\sharp J = [2, 10] -^\sharp [2, 4] = [-2, 8].$$

Hence

$$\text{back}_I^\sharp(x - y, [2, 4], A) = [x \mapsto [2, 10], y \mapsto [0, 8]].$$

Takeaways

1. Intervals are the canonical hull abstraction of integer sets:

$$\mathcal{P}(\mathbb{Z}) \begin{matrix} \alpha_I \\ \xleftrightarrow{\quad} \\ \gamma_I \end{matrix} \mathbb{I}.$$

2. A Galois connection gives the best description $\alpha(c)$ and the best correct transformer

$$f^{\text{best}} = \alpha_D \circ f \circ \gamma_C.$$

3. Compositional evaluation and filtering are sound but may lose dependencies.

Next lecture

Soundness of `while`, then interval widening to force ascending iterations to converge in finite time.